

Congress Trading Signal

La couche données

Extraction, identification, enrichissement et validation honnête des déclarations boursières du Congrès américain — Chambre *et* Sénat, 2020–2026

90 487 transactions (Chambre **81 646** + Sénat **8 841**) reconstruites de bout en bout depuis les sources officielles (*House Clerk*, Sénat *eFD*), piste électronique **et** scannée (OCR Claude Vision) ; chaque ligne rattachée à son auteur (`bioguide_id`), enrichie (ticker, secteur GICS→ETF, montant, ancienneté) jusqu'à la **table 12/12 champs**, validée contre Quiver (vérification externe, jamais réinjectée), et **reproductible** (golden octet-à-octet). Ce document se lit *main dans la main* avec le dépôt : chaque mécanisme cite la fonction Python réelle qui l'implémente.

Alice Lemaire

27 juin 2026

Résumé

Les membres du Congrès américain déclarent leurs transactions boursières sous le *STOCK Act* ; ces déclarations, exploitées par une stratégie de *copy-trading*, exigent au préalable une couche données propre, traçable et honnêtement validée. Ce rapport raconte la construction de cette couche, étape par étape — de la résolution d'identité jusqu'à la table finale enrichie. Le pipeline reconstruit **90 487 transactions** sur deux chambres et sept ans (2020–2026) : **81 646** pour la Chambre (32 676 électronique + 48 970 OCR) et **8 841** pour le Sénat (7 161 électronique + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (**99,99 %** des lignes Chambre, **100 %** au Sénat), enrichie (ticker, secteur **GICS→ETF SPDR**, type d'actif, montant *midpoint*, ancienneté), aboutissant à la **table 12/12 champs**. La validation externe contre Quiver — **jamais réinjectée** — donne une concordance transaction-niveau de **85,9 %/an** (Chambre, un plancher) et **98–100 %/an** (Sénat), avec **9 et 0** vrais-absents après explication ; le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne. La correction est figée par un **golden octet-à-octet** (108 fichiers Chambre, 69 Sénat) et des reproductions fonction-par-fonction. Le rapport rend compte, à chaque étape, du **problème** concret, de la **façon dont il est résolu** (avec la fonction réelle) et du **résultat chiffré**, sans masquer les limites.

Table des matières

Résumé	1
1 Introduction & contexte	3
2 Sources & acquisition	3
3 Architecture & méthodologie	3
3.1 Trois couches de code + une couche de données	3
3.2 Le pipeline : un point d'entrée unique	4
3.3 La convention de numérotation <i>est</i> l'enchaînement	4
3.4 Le voyage d'une transaction (vue compacte)	5
4 Construction de la donnée, étape par étape	5
4.1 D'abord, savoir qui a déclaré : la résolution d'identité	5
4.2 Ensuite, trier les formats : électronique ou scanné ?	6
4.3 La piste digitale de la Chambre (PDF lisibles)	6
4.4 La piste digitale du Sénat (HTML eFD)	6
4.5 La piste scannée de la Chambre (OCR Claude Vision)	7
4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée	7
4.7 Donner un secteur à chaque ticker (GICS → ETF SPDR)	7
4.8 La table finale : le contrat « 12/12 »	8
5 Le rapport de qualité	8
6 La validation externe (Quiver)	10
7 Assurance qualité & reproductibilité	10
8 Hypothèses explorées puis révisées	10
9 Limites (honnêteté)	11

10 Périmètre & suite	11
11 Conclusion	11
12 Annexes	12
12.1 Annexe A — inventaire complet des fonctions Python	12
12.2 Annexe B — les 44 fichiers .py en un coup d’œil	25
12.3 Annexe C — le schéma FINAL (28 colonnes)	26
12.4 Annexe D — valeurs vérifiées (recalculées depuis les FINAL)	27
12.5 Annexe E — reproduire / lancer	27

1 Introduction & contexte

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L'intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l'information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (anti-*look-ahead* : on date chaque transaction par sa date de divulgation, jamais par une date postérieure connue).

Avant toute stratégie, il faut une **couche données** irréprochable : extraire fidèlement les déclarations des deux chambres, les rattacher à la bonne personne, les enrichir (ticker, secteur, montant) et les valider honnêtement. C'est l'objet de ce rapport. La stratégie et le backtest associés constituent un travail de recherche séparé, hors du périmètre traité ici.

2 Sources & acquisition

Principe anti-look-ahead. Chaque transaction est datée par sa **date de divulgation** (`disclosure_date`, la date de dépôt du PTR), jamais par une date postérieure connue : une stratégie ne peut copier qu'*après* publication.

Chambre — House Clerk. Un index XML par année (`{an}FD.xml`) liste les dépôts ; on filtre `FilingType=P` (les PTR) et on télécharge le PDF de chaque dépôt. Source publique, sans barrière. Les PDF se répartissent en **lisibles** (couche texte → parsing déterministe) et **scannés** (image → OCR).

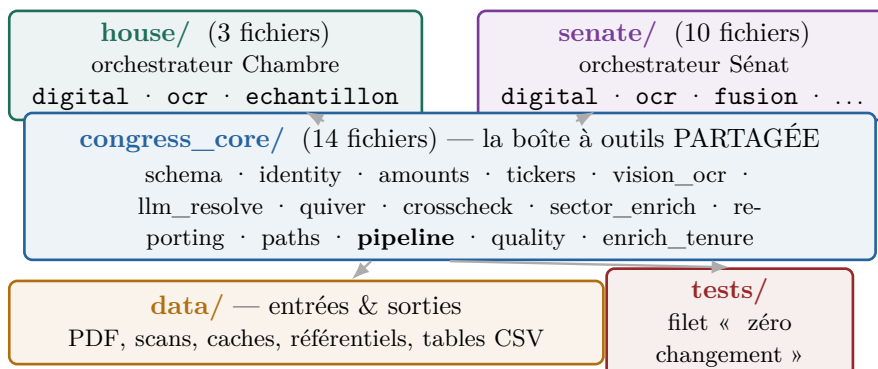
Sénat — eFD (*Electronic Financial Disclosure*). Plus fermé : un **garde-fou CSRF**. Il faut d'abord accepter un formulaire d'accord (`accept_agreement` : GET pour un `csrftoken`, puis POST `prohibition_agreement` avec `csrftoken` et l'en-tête `X-CSRFToken`). On interroge ensuite l'API de recherche (`report_types=[11]` = PTR). Les déclarations électroniques sont en **HTML** ; les déclarations papier renvoient des **images .gif** servies par `efd-media-public.senate.gov`. On reste « poli » (pause entre requêtes, aucun contournement) et on **cache tout sur disque** — un re-run ne re-télécharge rien.

Quiver = vérification externe *uniquement*. Quiver Quantitative (agrégateur commercial) sert de **vérité-terrain indépendante**. Il n'est *jamais* réinjecté dans nos tables : on mesure si notre extraction concorde, on ne la complète pas. C'est une règle structurante du projet.

3 Architecture & méthodologie

3.1 Trois couches de code + une couche de données

La structure est une **boîte à outils partagée** (`congress_core`, 14 modules) que **deux orchestrateurs** (un par chambre) utilisent, plus la donnée et les tests.

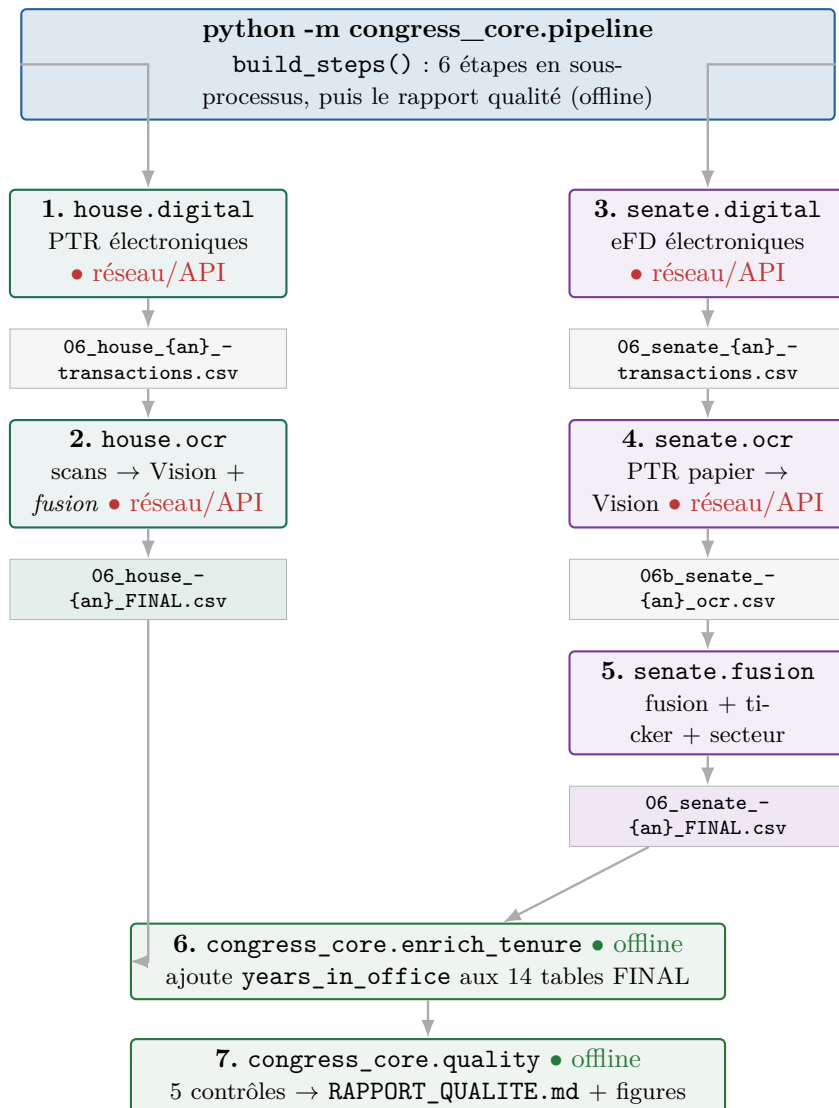


L'idée directrice

`congress_core` = les **ingrédients et ustensiles** (couper un nom en bioguide, lire un scan, calculer un secteur...). `house/` et `senate/` = **deux recettes** qui s'en servent dans leur ordre. Elles ne diffèrent que parce que les **sources** diffèrent : Chambre = index XML + PDF du *Clerk* ; Sénat = eFD (HTML) + images .gif. D'où plus de fichiers côté Sénat (scraping et OCR papier plus complexes).

3.2 Le pipeline : un point d'entrée unique

`congress_core/pipeline.py` (fonction `build_steps`) enchaîne les modules en sous-processus. La commande `python -m congress_core.pipeline -years 2020-2026` assemble **6 étapes** d'orchestration ; un **7^e** traitement — le rapport qualité (`congress_core/quality.py`) — se lance séparément, hors-ligne et en lecture seule.



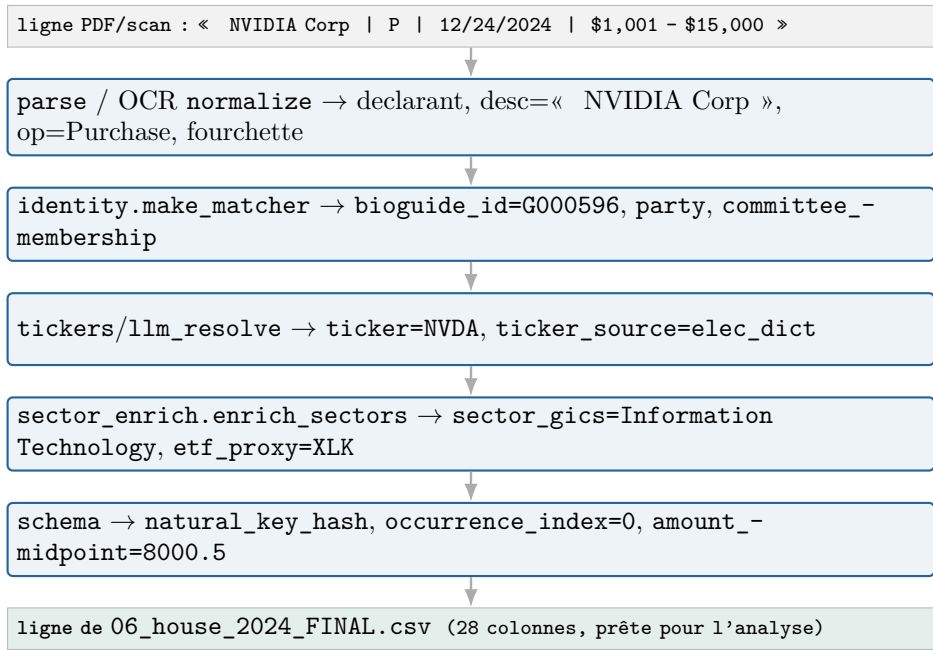
3.3 La convention de numérotation est l'enchaînement

Sur le disque, le préfixe d'un fichier = son étape dans le pipeline. C'est ce qui rend la couche données lisible sans documentation externe.

Préfixe	Étape	Écrit par
03_	index des PTR (<code>FilingType=P</code>) dans la fenêtre	house.digital
04_	manifeste : lisible / non_lisible / absent	house.digital
05_	échecs de parsing	*.digital
06_	table digitale (PTR électroniques)	*.digital
06b_	table OCR (scans / papier)	*.ocr
06c_	échecs OCR	*.ocr
06_..._FINAL	digital + OCR fusionnés , 28 colonnes	house.ocr / senate.fusion
07_...07f_	comparaison / réconciliation Quiver (validation)	quiver / quiver_- audit
08_	cross-check « semaine 1 » (Chambre)	house.digital
00_	tableaux de bord (statut, concordance)	*.digital / fusion
_préfixe	caches & brouillons (<code>_quiver_*_cache</code> , <code>_scan_census...</code>)	divers

3.4 Le voyage d'une transaction (vue compacte)

Avant le récit détaillé (§4), voici la trajectoire d'une ligne, de la source à la table finale — en nommant la fonction à chaque étape.



4 Construction de la donnée, étape par étape

Cette section suit une transaction *dans l'ordre où le pipeline la construit* — du nom griffonné sur un formulaire jusqu'à la ligne enrichie de la table finale. Pour chaque étape : le **problème** concret, **comment on le résout** (avec la fonction réelle), et le **résultat** chiffré.

4.1 D'abord, savoir qui a déclaré : la résolution d'identité

Le problème. Une déclaration ne donne qu'un *nom libre* — « Hon. Earl L. Carter » ici, « Buddy Carter » là, « N. Pelosi » ailleurs. Sans clé stable, on compterait deux fois la même personne et on ne pourrait rattacher ni son parti ni ses commissions.

Comment on le gère. Le module `congress_core/identity.py` charge le référentiel public *congress-legislators* (`load_reference`, live avec `repli YAML`), et construit un index de noms enrichi de variantes (surnom, second prénom, premier mot du nom officiel, dernier mot d'un nom composé) ainsi que

l'année de première entrée en fonction. Le cœur est la cascade `make_matcher`, qui résout (`nom`, `prénom`) dans l'ordre : (1) **override manuel** (un seul cas figé, « Nicholas V. Taylor » = Van Taylor); (2) **correspondance exacte**, en testant aussi le surnom (table de 31 surnoms : richard→rick...) et le second prénom; (3) **repli par nom de famille unique**. `enrich_identity` ajoute ensuite `party`, `committee_membership` et le drapeau de commission clé. Côté Sénat, `senate/identity.py` ajoute une désambiguïsation par chambre/titulaire et exclut les **délégués** non-votants.

Le résultat. 99,99 % des lignes Chambre (81 642 / 81 646) et 100 % du Sénat reçoivent un `bioguide_id`. Les 4 lignes sans identifiant sont *une* déclaration manuscrite de 2023 (« Ada Norah Henriquez », doc 8219483). On compte **256 bioguides pour 275 noms** (Chambre) : l'écart, ce sont les variantes orthographiques d'un même élu, rattachées au même identifiant — exactement ce que la résolution corrige.

4.2 Ensuite, trier les formats : électronique ou scanné ?

Le problème. Une déclaration arrive sous deux formes très différentes, qui n'appellent pas le même traitement : un **texte** exploitable directement, ou une **image** qu'il faut « lire ».

Comment on le gère. Côté Chambre, `house/digital.py:build_manifest` ouvre chaque PDF avec `pdfplumber` et applique un test binaire simple — `bool(text.strip())` : une couche de texte non vide ⇒ **lisible** (piste électronique); sinon ⇒ **non_lisible** (piste OCR); fichier absent ⇒ **absent**. Le verdict est écrit dans `04_download_manifest.csv` et **pilote tout le reste**. Côté Sénat, la distinction vient de la liste eFD elle-même (`fetch_ptr_list`) : chaque dépôt porte un `kind` — `ptr` (déclaration HTML électronique) ou `paper` (image .gif scannée).

Le résultat. Ce tri d'apparence anodine est la fourche du pipeline. Il explique la composition finale : Chambre 32 676 électronique + 48 970 OCR; Sénat 7 161 électronique + 1 680 papier.

4.3 La piste digitale de la Chambre (PDF lisibles)

Le problème. Sur un PDF Clerk lisible, les transactions ne sont pas un tableau propre : les montants débordent parfois sur deux lignes, le ticker se cache entre parenthèses, l'actif s'étale sur plusieurs lignes de continuation.

Comment on le gère. `house/digital.py:run_year` enchaîne : `load_ptr_index` (index XML, `FilingType=P`, fenêtre de divulgation → `03_ptr_index.csv`); `parse_ptr`, qui détecte chaque transaction par l'expression régulière `TXN_RE` (`P|S|E`, dates, fourchette \$), recolle les lignes coupées (`_join_split_lines`) et remonte le contexte d'actif/ticker sur quelques lignes; puis `join_identity` (rattache bioguide + métadonnées) et `finalize` (clé naturelle, `occurrence_index`, dédup non destructrice). La sortie est `06_house_{an}_transactions.csv`; les PDF lisibles ne donnant aucune ligne sont tracés dans `05_parse_failures.csv`.

Le résultat. 32 676 transactions électroniques, avec un taux de parsing très élevé (99–100 %); les rares échecs sont *tracés*, jamais silencieux.

4.4 La piste digitale du Sénat (HTML eFD)

Le problème. Le Sénat n'expose pas de PDF : il faut franchir un **garde-fou CSRF** puis parser des **tableaux HTML** dont les colonnes varient d'une déclaration à l'autre.

Comment on le gère. `senate/digital.py:run_year` commence par `accept_agreement` (GET du `csrftoken`, POST `prohibition_agreement` avec l'en-tête `X-CSRFToken`), puis `fetch_ptr_list` interroge l'API de recherche (`report_types=[11]`) et `fetch_report` récupère chaque déclaration (cache disque). `parse_electronic` lit alors les tableaux via `pd.read_html`, repère les bonnes colonnes par appariement flou (`_find_col`), normalise l'opération (`norm_op`) et le montant (`amount_midpoint`). L'enrichissement identité/ticker/dédup est délégué à `senate/identity.py:enrich`. La sortie est `06_senate_{an}_transactions.csv`.

Le résultat. 7 161 transactions électroniques pour le Sénat.

4.5 La piste scannée de la Chambre (OCR Claude Vision)

Le problème. Reste l'autre moitié : des **images**. À quoi ressemblent-elles ? Le census des 547 PDF scannés en distingue trois familles : **A** — **tapé droit** (74 docs), **B** — **tapé mais l'image est couchée** (322 docs, le gros du volume), **C** — **manuscrit** (151 docs). Une page couchée, un modèle de vision la lit mal ; une date manuscrite, il la lit mal aussi.

Comment on le gère. Le moteur `congress_core/vision_ocr.py` **redresse d'abord** : `detect_rotation` montre la page aux 4 orientations (`ROT_CANDIDATES=[0,90,180,270]`) et fait *reconnaître* la bonne, puis `rotate_b64_png` la remet droite. `VisionExtractor._call` extrait ensuite en JSON structuré (`tool_choice` forcé sur `record_transactions`, `claude-sonnet-4-6`, 7 tentatives), avec un **cache versionné** par `prompt_sha` (`extract_cached`) qui ne re-paie que les lots en erreur. Le ticker absent du papier est **ré-associé** via le dictionnaire des symboles vus en électronique, complété par une passe LLM (`house/ocr.py:llm_resolve_tickers`). **Point d'architecture** : côté Chambre, la **fusion digital+OCR**, le **ticker** et le **secteur** se font **ANNÉE PAR ANNÉE**, pendant le run (`house/ocr.py:run_ocr_year` écrit `06_house_{an}_FINAL.csv`), parce que le volume y est énorme. Le cluster C manuscrit est **exclu par défaut** (dates trop incertaines), avec récupération ciblée.

Le résultat. **48 970** lignes OCR. Le volume est très concentré : Rohit Khanna en représente **62,9 %** à lui seul, le top 3 (Khanna, McCaul, Harshbarger) **92,4 %** — ce sont de gros déposants qui déclarent exclusivement sur papier.

4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée

Le problème. Côté Sénat, le papier est marginal (5 sénateurs) mais bien réel : des images `.gif`. Et surtout, contrairement à la Chambre, les données sont *peu nombreuses*.

Comment on le gère. `senate/ocr.py` lit les `.gif` par Vision et écrit `06b`. Puis — et c'est là l'unique vraie asymétrie d'architecture entre les deux chambres — `senate/fusion.py:main` procède en deux temps : **Étape 1**, fusion non destructrice *par année* (digital + OCR, dédup sur (`natural_key_hash`, `occurrence_index`)); **Étape 2**, enrichissement sur le **corpus complet en une seule passe** — `ticker_resolve.resolve_tickers(big)` puis `enrich_sectors(big)` sur *toutes* les années concaténées, avec « dict + LLM partagés, cache unique ». **Pourquoi cette différence avec la Chambre ?** La Chambre a assez de volume pour traiter chaque année isolément ; le Sénat, lui, est petit et chargé en papier — **mutualiser le dictionnaire de tickers sur tout le corpus** (un symbole vu une année sert aux autres) donne une bien meilleure couverture. D'où un module `fusion.py` dédié, exécuté *après* que toutes les années sont assemblées, là où la Chambre fond et enrichit *pendant* chaque année.

Le résultat. **1 680** lignes papier, très concentrées sur Blumenthal (~73%). C'est aussi ce papier (obligations municipales, *munis*) qui explique la baisse de couverture ticker/secteur du Sénat en 2025–26 (g5).

4.7 Donner un secteur à chaque ticker (GICS → ETF SPDR)

Le problème. Une stratégie en ETF a besoin du **secteur**, pas seulement du ticker ; or certains tickers sont délistés ou introuvables.

Comment on le gère. `congress_core/sector_enrich.py:enrich_sectors` résout en hybride : `yfinance` (factuel) → repli **LLM** (énumération GICS stricte) → overrides manuels, en traçant qui a tranché (`sector_source`). Le secteur GICS est mappé 1 : 1 vers un **ETF Select Sector SPDR** (Energy XLE, Materials XLB, Industrials XLI, Cons. Disc. XLY, Cons. Staples XLP, Health Care XLV, Financials XLF, IT XLK, Comm. Services XLC, Utilities XLU, Real Estate XLRE). Au passage, les montants déclarés en **fourchettes** sont ramenés à leur **milieu** (`amounts.py:amount_midpoint`), et l'ancienneté `years_in_office` est calculée hors-ligne et *appendue* en dernière étape (`enrich_tenure.py`, octet-préservant, idempotent).

Le résultat. Secteur renseigné à **83,2 %** (Chambre) et **62,1 %** (Sénat) ; ancienneté à **100 %** sur les deux chambres.

4.8 La table finale : le contrat « 12/12 »

Le liant — le cœur partagé. Ce qui assemble tout sans rien perdre, c'est `schema.py` : une **clé naturelle** de 7 champs (*sans* le ticker, pour rester stable à l'enrichissement), numérotée par `occurrence_index` (le rang d'un lot répété *dans un même dépôt*), et une **déduplication non destructrice** (`dedup_canonical`) qui retire les vrais doublons cross-dépôt mais **préserve les lots multi-comptes réels** — par exemple les déclarations répétées de Khanna via plusieurs comptes.

Le contrat. La table FINAL compte **28 colonnes** (27 d'assemblage + `years_in_office`). Les **12 champs « métier »** exigés sont *exactement* la constante `schema.CORE_FIELDS` :

```
declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date  
· ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type
```

Nuance d'honnêteté : « 12 champs *présents* » $\neq$ « 12 champs *sans valeurs manquantes* »

Les 12 champs sont **présents** dans le schéma des deux chambres. Mais trois ont des **trous légitimes**, par nature et non par défaut d'extraction : `ticker/sector_gics/etf_proxy` sont vides pour les actifs non cotés (obligations, *munis*, fonds privés) ; `committee_membership` est un **snapshot courant** (pas l'historique daté). Les champs structurellement toujours renseignables (nom, chambre, parti, dates, opération, montant) sont quasi-complets. Nous assumons cette distinction.

Taux de remplissage (rows-based, recalculés depuis les FINAL) : `ticker` 85,3 % / 71,4 % ; `sector_gics=etf_proxy` 83,2 % / 62,1 % ; `asset_type` 91,1 % / 97,3 % ; `committee_membership` 86,6 % / 65,6 % (Chambre / Sénat). Le schéma complet des 28 colonnes (sens + origine) est en annexe 12.3.

5 Le rapport de qualité

Le problème. Une table « pleine » n'est pas une table *saine* : les dates peuvent être incohérentes, les délais légaux non respectés, la couverture concentrée sur quelques personnes.

Comment on l'obtient. `congress_core/quality.py` (`build_report` $\rightarrow$ `_figures`) calcule **5 contrôles** et génère 4 figures — **en lecture seule des FINAL, sans aucun appel API** (matplotlib Agg). (a) **Cohérence des dates** (`disclosure` $\geq$ `transaction`) : 99,8 % Chambre, 100,0 % Sénat. (b) **Délai légal** : 86,9 % (Chambre) et 84,9 % (Sénat) des dépôts dans les 45 jours (médiane **28 j**). (c) **Montants** : médiane **8 000 \$**, quartile haut **32 500 \$**. (d) **Couverture par membre** : 320 déposants, dont **206** avec ≥ 10 transactions (éligibles) et **150** actifs sur ≥ 3 années. (e) **Achats sans sortie déclarée** : 22,5 % (Chambre) / 39,6 % (Sénat).

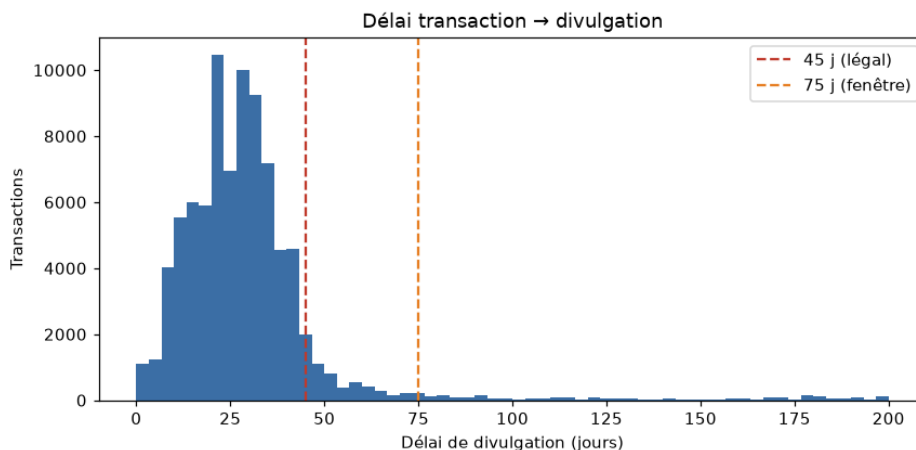


FIGURE 1 – Délai disclosure – transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `congress_core/quality.py` (`_figures`, source `delay_buckets`) — lecture seule, aucun appel API.

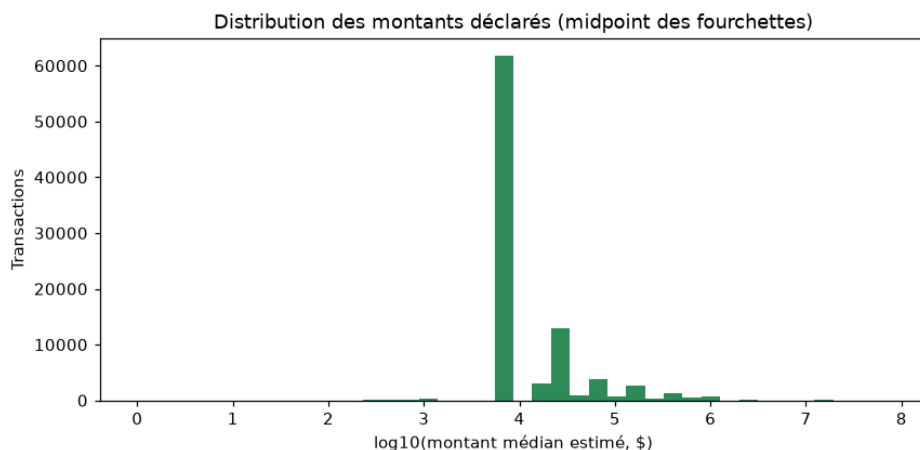


FIGURE 2 – Distribution de `amount_midpoint` en échelle `log10`. Générée par `quality.py` (`_figures`, source `amount_distribution`).

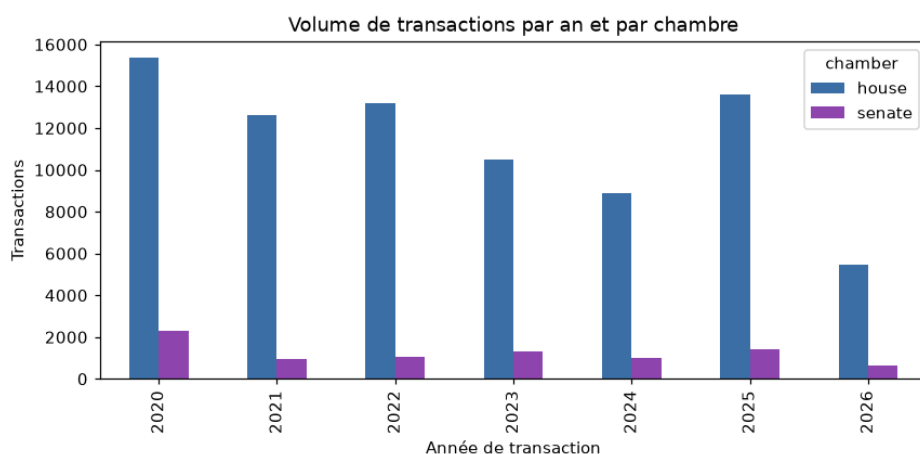


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `quality.py` (`_figures`).

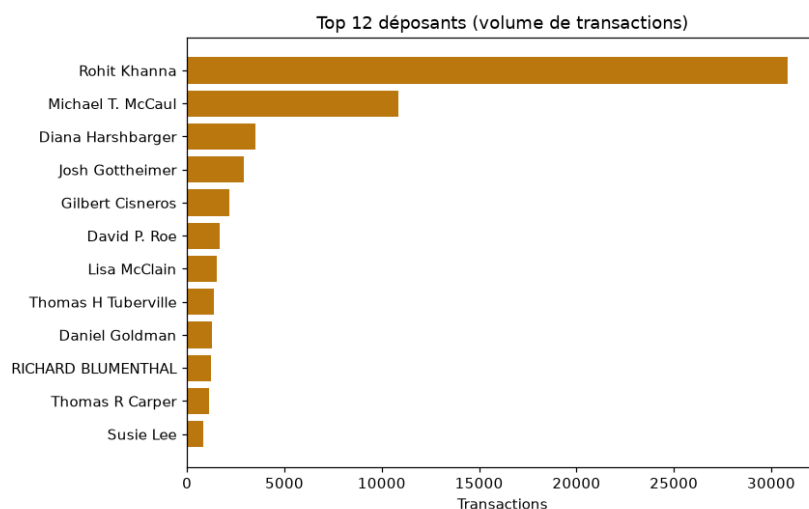


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `quality.py` (`_figures`, source `coverage_per_member`) — la domination des déposants-papier (Khanna, McCaul) y est visible.

La couverture par an — et la baisse Sénat 2025–26. Le global masque une dynamique : au Sénat, la couverture ticker chute à **45,4 %** (2025) et **43,7 %** (2026), contre 80 % les années antérieures ; le secteur suit (37 % vs 70 %). **Pourquoi ?** La part de papier OCR et surtout d'**obligations municipales** (non cotées, donc `ticker=None` *légitimement*, ex. Blumenthal) y augmente fortement. Ce n'est pas une dégradation d'extraction mais un changement de composition des actifs ; le global (71,4 % / 62,1 %) est tiré vers le haut par les années antérieures. La fiabilité des dates OCR est suivie par `date_confidence` (fenêtre 75 j, colonne *OCR seulement*) : 95,3 % plausibles côté Chambre — le creux 2021 (79,6 %) est entièrement Diana Harshbarger (manuscrit dense), une anomalie localisée, non systémique.

6 La validation externe (Quiver)

Le problème. Comment savoir si l'extraction est juste, *sans* se contenter de se croire ?

Comment on le gère. On compare le FINAL à Quiver sur **deux axes**, sans jamais le réinjecter : (1) **comptes par déposant** (per-lot) ; (2) **recouvrement transaction-niveau** via la clé bioguide | ticker | date | type (la date appariée est le *Traded* de Quiver). La **couverture** = transactions Quiver retrouvées / total Quiver.

Le résultat — Chambre (clé « brute », sans tolérance de date, `quiver.validate_quiver_house`) : couverture globale **85,9 %** (43 136 / 50 224), par année de 80,7 % à 91,3 %. C'est un **plancher** : la clé sur date brute sous-estime sur les dates OCR bruitées. Les **249 « manquants bruts »** se réduisent à **9 vrais-absents** après explication des artefacts (tolérance ± 1 j, amendements, options/obligations) — **aucun bug d'extraction**. **Sénat** (clé normalisée, `quiver.audit.reconcile`) : couverture **98–100 %/an**, accord de sens $\sim 95,7 %$, montant $\sim 93,2 %$, **554 amendements** dédupliqués, **0 vrai raté** après audit adversarial.

Le constat « OCR = source unique »

Quiver — comme Kadoa et *House Stock Watcher* — **ne voit aucun scan** : 0 ligne pour les gros déposants-papier (Khanna, Harshbarger, Blumenthal). Le terme `only_ours` de la Chambre ($\sim 16\,000$ transactions) est donc surtout le **papier que les agrégateurs ne voient pas** — pas une erreur, mais notre **valeur ajoutée**, validée en interne (cohérence, stabilité run-à-run, `date_confidence`). `crosscheck.py` matérialise ce statut par déposant.

7 Assurance qualité & reproductibilité

Le problème. Le pipeline n'est **pas rejouable hors-ligne** (le scraping exige le réseau, l'OCR l'API) ; rejouer une année digitale produirait une table *vide* (les PDF lisibles ont été parsés puis écartés).

Comment on le gère. On prouve la correction par **reproduction depuis les colonnes figées**, preuve plus forte car déterministe : (i) un **golden octet-à-octet** — empreintes SHA-256 de toutes les sorties, **108** fichiers Chambre et **69** Sénat (`check_golden.py` / `senate_check_golden.py` doivent afficher « ZÉRO ÉCART ») ; (ii) des **reproductions fonction-par-fonction** (`test_senate_repro` reconstruit `natural_key_hash` 8841/8841, l'identité 67/67, le ticker 65/65 ; côté Chambre `test_schema/test_amounts_tickers/` `test_identity/test_tenure`) ; (iii) `test_vision_sha/test_incremental` (le `prompt_sha` est stable, un 2^e run OCR fait 0 appel) ; (iv) `audit_metrics.py`, qui recompute toutes les métriques et *assure* les invariants — la source de vérité chiffrée de ce rapport.

8 Hypothèses explorées puis révisées

La valeur « recherche » tient autant aux choix *révisés* qu'aux résultats. Pour chacun : **hypothèse de départ** → **problème constaté** → **décision finale**.

Échantillon d'abord. Avant le run complet des 547 PDF, `house/echantillon.py` a OCR-isé un **échantillon stratifié** des clusters A/B/C pour *mesurer* la qualité et durcir le prompt — plutôt

que de découvrir les modes d'échec en production.

Deskew OCR. *Départ* : demander au modèle « de combien faut-il tourner ? ». *Problème* : tâche spatiale peu fiable — il répond presque toujours « 90 ». *Décision* : lui montrer la page aux **4 rotations** et lui faire *reconnaître* la droite. La concordance OCR passe de **59,7 % à ≈69 %**.

Déduplication. *Départ* : déduplicer sur la seule clé naturelle. *Problème* : cela écrase des **lots multi-comptes réels** (Khanna déclarant plusieurs fois le même titre). *Décision* : `occurrence_index` → **dédup non destructrice** sur le couple.

Cluster C manuscrit. Tenté, puis **exclu** par défaut (dates trop incertaines pour une stratégie datée). Une sonde **Opus vs Sonnet** sur C a confirmé qu'Opus **ne corrige pas le goulot** (les dates), pour un coût $\approx \times 3$ — on garde Sonnet, C reste exclu.

Fenêtre de plausibilité. Le STOCK Act vise ≈ 45 j ; on tolère **75 j** (`date_confidence`) pour absorber le bruit OCR — colonne calculée *sur l'OCR seulement*.

Ticker en trois voies. Ajoutées par échecs successifs : explicite, dictionnaire électronique, puis LLM (filtre `is_equity`, pour ne *jamais* tickeriser une obligation/muni).

Quiver, deux régimes de clé. Chambre = clé « brute » (couverture = plancher) ; Sénat = clé « normalisée » (*reconcile*) avec **dédup** des amendements. On *n'unifie pas* les clés : chaque chambre garde sa sémantique exacte.

Per-année (Chambre) vs corpus (Sénat). L'enrichissement se fait *pendant* chaque année côté Chambre (volume), mais *sur tout le corpus en une passe* côté Sénat (pour mutualiser le dictionnaire de tickers) — cf. §4.

9 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** (dates trop incertaines) — choix assumé, pas une lacune cachée.
- **Validation papier externe impossible** : aucun agrégateur ne voit les scans → pas de taux d'exactitude *externe* sur le papier (seulement des contrôles internes).
- **Appariement Quiver Chambre sans tolérance de date** : la couverture **85,9 %** est un **plancher**, pas une borne haute.
- **ticker/sector_gics/etf_proxy structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut.
- **Commissions non « point-in-time »** : `committee_membership` est un *snapshot* courant, pas l'historique daté.
- **date_confidence = heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** ; ce n'est pas une vérité-terrain.

10 Périmètre & suite

La **couche données** décrite ici est livrée : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest** associés (copy-trading actions puis ETF sectoriels) constituent un travail de recherche séparé, hors-périmètre de ce rapport.

11 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est identifiée (99,99 / 100 %), enrichie (table 12/12 sur les deux chambres) et validée contre Quiver sans jamais le réinjecter (85,9 % Chambre, 98–100 % Sénat ; 9 et 0 vrais-absents). Le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne ; la cor-

rection est figée (golden octet-à-octet) et reproductible. Surtout, le rapport assume ses nuances — « présents » n’est pas « sans manquants », la couverture Sénat 2025–26 baisse pour des raisons de composition d’actifs, et la couverture Quiver Chambre est un plancher. C’est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

12 Annexes

12.1 Annexe A — inventaire complet des fonctions Python

Les 228 fonctions et méthodes réelles des 27 modules de code (les 3 `__init__.py` sans fonction sont omis). Chaque entrée décrit le **corps réel**, ce que la fonction **renvoie**, et l’**étape** du pipeline. Inventaire recoupé contre l’arbre syntaxique (AST) : aucune fonction réelle n’est omise.

Cœur partagé — `congress_core/` (14 modules, 101 fonctions)

schema.py — *Schéma canonique unifié, clé naturelle 7 champs, hash et déduplication non destructrice partagés House/Sénat.*

Fonction	Ce qu’elle fait (corps réel)	Renvoie	Étape
<code>natural_key</code>	Concatène avec « » les 7 champs (str de chaque), chamber pris de la ligne sinon de l’argument, jamais codé en dur.	<i>str : représentation textuelle stable de la clé</i>	dédup/schéma
<code>natural_key_hash</code>	Encode en UTF-8 la clé naturelle puis calcule son SHA-256 hexadécimal.	<i>str : empreinte SHA-256 hexadécimale</i>	dédup/schéma
<code>add_occurrence_index</code>	Copie le DataFrame, ajoute <code>occurrence_index</code> = rang cumcount par groupe (<code>doc_id</code> , <code>natural_key_hash</code>) pour préserver les lots répétés.	<i>DataFrame avec colonne occurrence_index</i>	dédup/schéma
<code>dedup_canonical</code>	Trie par <code>disclosure_date</code> décroissant puis garde une ligne par (<code>natural_key_hash</code> , <code>occurrence_index</code>), retirant les doublons cross-dépôt.	<i>DataFrame déduplicé canonique</i>	dédup/schéma

identity.py — *Identité : normalisation de noms, référentiel des législateurs/commissions, résolution bioguide et ancienneté.*

Fonction	Ce qu’elle fait (corps réel)	Renvoie	Étape
<code>strip_accents</code>	Normalise en NFKD et retire les caractères combinants pour ôter les accents d’une chaîne.	<i>str : chaîne sans accents</i>	identité
<code>norm</code>	Retire accents, passe en minuscules, remplace tout non-alphabétique par des espaces, réduit les espaces.	<i>str : nom normalisé</i>	identité
<code>split_name</code>	Découpe « Prénom ... Nom[, suffixe] » : prend la partie avant la virgule, ôte les suffixes, renvoie dernier, premier et milieux.	<i>tuple (last, first, liste des milieux)</i>	identité
<code>Reference.__init__</code>	Stocke les structures du référentiel : universe, index de noms, commissions, <code>key_bios</code> , bios courants, source, première année.	— (<i>effet de bord : initialise l’objet</i>)	identité
<code>Reference.party</code>	Renvoie le parti du bioguide depuis <code>ref_universe</code> si présent dans l’index, sinon None.	<i>str parti ou None</i>	identité
<code>Reference.years_in_office</code>	Calcule <code>as_of_year</code> moins la première année en fonction ; None si données manquantes ou résultat négatif.	<i>int ancienneté ou None</i>	ancienneté
<code>Reference.committees</code>	Joint par « ; » les commissions triées du bioguide, ou chaîne vide si bioguide absent.	<i>str : commissions ou vide</i>	identité
<code>Reference.is_key_committee</code>	Teste l’appartenance du bioguide à l’ensemble <code>key_bios</code> ; False si bioguide vide.	<i>bool : membre d’une commission clé</i>	identité

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_last_term_key</code>	Extrait l'année de fin du dernier mandat et la renvoie négative pour tri ; valeurs spéciales si absente ou illisible.	<i>int : clé de tri par fin de mandat</i>	utilitaire
<code>_load_people</code>	Tente de télécharger législateurs current et historical en live, repli sur YAML embarqué en cas d'échec.	<i>tuple (cur, hist, source)</i>	acquisition
<code>_load_committees</code>	Télécharge commissions et appartenances en live, repli sur YAML embarqué si échec ou non-live.	<i>tuple (committees, membership)</i>	acquisition
<code>load_reference</code>	Construit le référentiel : trie les personnes, indexe variantes de noms (surnom, milieu, composé), première année, commissions, key_bios.	<i>objet Reference assemblé</i>	identité
<code>make_matcher</code>	Fabrique une closure <code>match(last, first)</code> : override manuel, candidats nom/prénom avec surnoms, lookup exact, repli nom unique.	<i>fonction match(last, first) → bioguide ou None</i>	identité
<code>enrich_identity</code>	Copie le DataFrame, mappe bioguide via matcher (colonnes last/first ou split_name), ajoute chamber, party, commissions, key_flag.	<i>DataFrame enrichi en identité</i>	identité
<code>add_years_in_office</code>	Copie le DataFrame, extrait l'année de transaction (repli divulgation) et calcule years_in_office par bioguide.	<i>DataFrame avec colonne years_in_office</i>	ancienneté

amounts.py — Fourchettes de montant, midpoint, propriétaire et type d'opération, maps indexées par chambre sans fusion.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>amount_midpoint</code>	Extrait les nombres « \$X » de la chaîne : moyenne des deux premiers, sinon le seul nombre, sinon None.	<i>float midpoint ou None</i>	parsing montant
<code>operation_type_from_code</code>	Mappe un code en majuscules : P→Purchase, E→Exchange, sinon Sale avec variante Partial/Full selon le sous-type.	<i>str : type d'opération 5-voies</i>	parsing

tickers.py — Normalisation et récupération de ticker, inférence du type d'actif par chambre, copies exactes des moteurs.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>recover_ticker</code>	Si la description correspond au motif « le nom EST un ticker », renvoie le symbole capté, sinon None.	<i>str ticker ou None</i>	ticker
<code>explicit_ticker</code>	Cherche un symbole explicite en fin de description, ôte les points, renvoie s'il fait 1 à 5 caractères.	<i>str ticker ou None</i>	ticker
<code>norm_asset</code>	Met en majuscules, retire ticker explicite et ponctuation, supprime les suffixes corporatifs, applique des corrections OCR.	<i>str : description normalisée pour lookup</i>	ticker
<code>_infer_asset_type_house</code>	Teste en cascade des regex House (gouvernemental, structuré, fonds, obligation, action) sur la description majuscule.	<i>str type d'actif ou None</i>	secteur
<code>_infer_asset_type_senate</code>	Teste en cascade des regex Sénat (municipal, obligation, autre, action) sur la description majuscule.	<i>str type d'actif ou None</i>	secteur
<code>infer_asset_type</code>	Aiguille vers la variante Sénat ou House selon le paramètre chamber pour inférer le type d'actif.	<i>str type d'actif ou None</i>	secteur

vision_ocr.py — Moteur OCR Claude Vision chambre-agnostique : rendu image, deskew, appel tool_use et cache versionné avec reprise.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
OcrError	Classe d'exception dédiée aux échecs OCR ; corps vide (juste pass).	— (<i>classe d'exception, pas de return</i>)	utilitaire
rotate_b64_png	Décode le PNG b64, le tourne de -angle via PIL (horaire), réencode ; retourne tel quel si multiple de 360.	<i>str : PNG base64 tourné</i>	OCR Vision (deskew)
detect_rotation	Montre les 4 rotations à Vision, lit le chiffre choisi par regex, retourne l'angle ; backoff puis 0 si échec.	<i>int : angle horaire (0 par défaut)</i>	OCR Vision (deskew)
pdf_to_b64_images	Rend chaque page PDF en PNG b64 via pymupdf ; si deskew, détecte et redresse chaque page en basse résolution.	<i>liste PNG b64, ou (images, angles) si deskew</i>	OCR Vision (rendu)
image_b64_from_url	Télécharge l'image (cache disque clé SHA1 de l'URL), la redimensionne sous long_edge, renvoie PNG b64.	<i>str : PNG base64 redimensionné</i>	OCR Vision (rendu)
VisionExtractor.__init__	Stocke modèle, prompt, tool, pipeline_tag et paramètres (max_tokens, images par appel) sur l'instance.	— (<i>effet de bord : initialise l'objet</i>)	OCR Vision (init)
VisionExtractor.prompt_sha	Propriété : calcule le SHA256 tronqué à 12 de prompt plus tool trié plus pipeline_tag pour versionner le cache.	<i>str : empreinte hex de 12 caractères</i>	OCR Vision (versionnage)
VisionExtractor.call	Un appel Vision avec tool_use forcé, extrait les transactions du bloc tool_use ; backoff sur 429/5xx jusqu'à max_retries.	<i>list : transactions extraites</i>	OCR Vision (appel API)
VisionExtractor.extract_cached	Lit le cache si version concorde, sinon batche les images et n'appelle que les batches manquants ou en erreur, puis écrit le JSON.	(<i>liste transactions, dict objet cache</i>)	OCR Vision (cache, reprise)

llm_resolve.py — Résolveur LLM générique à cache versionné ; factorise le motif tool_use et porte la passe House nom vers ticker.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
VersionedLLMCache._init__	Stocke le chemin, le modèle et le prompt_sha qui versionnent le cache disque.	— (<i>effet de bord : initialise l'objet</i>)	ticker (cache)
VersionedLLMCache.load	Lit le JSON disque ; renvoie les mappings seulement si modèle et prompt_sha concordent, sinon dictionnaire vide.	<i>dict : mappings, ou {} si version différente</i>	ticker (cache)
VersionedLLMCache.save	Écrit sur disque un JSON contenant modèle, prompt_sha et les mappings fournis.	— (<i>effet de bord : écrit le fichier cache</i>)	ticker (cache)
map_batch	Un appel tool_use forcé renvoyant l'input du bloc tool_use ; backoff sur 429/5xx, relève après max_retries.	<i>dict : input du tool, ou {} si absent</i>	ticker (appel LLM)
ticker_prompt_sha	Calcule le SHA256 tronqué à 16 de TICKER_PROMPT plus TICKER_TOOL trié plus modèle pour versionner le cache.	<i>str : empreinte hex de 16 caractères</i>	ticker (versionnage)
resolve_names_to_tickers	Pour les lignes sans ticker, interroge le LLM par lots (cache versionné), valide via regex/is_equality, puis remplit ticker et marque la source llm.	<i>DataFrame : df avec tickers complétés</i>	ticker (résolution)

quiver.py — Vérification externe Quiver (jamais réinjectée) : fetch cache-first, clés d'appariement et réconciliation House et Sénat.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
norm_ticker	Met en majuscules, vide les valeurs nulles, retire suffixe PUT/CALL, remplace points et tirets par underscore.	<i>str : ticker normalisé (vide si invalide)</i>	validation Quiver (normalisation)

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>norm_sense</code>	Réduit le sens à 3 valeurs selon la première lettre : p->Purchase, s->Sale, e->Exchange, sinon brut ou point d'interrogation.	<i>str : sens canonique 3-voies</i>	validation Quiver (normalisation)
<code>low_bound</code>	Extrait par regex le premier montant en dollars de la fourchette et le convertit en entier sans virgules.	<i>int : borne basse, ou None si absent</i>	validation Quiver (montant)
<code>fetch_quiver</code>	Charge le cache CSV s'il existe, sinon appelle l'API avec le token, filtre par chambre et renomme les colonnes ; None si pas de token.	<i>DataFrame Quiver filtré, ou None</i>	validation Quiver (fetch)
<code>validate_quiver_house</code>	Filtre Quiver sur la fenêtre, compare les comptes par déclarant brut, recoupe au niveau transaction par clé brute, calcule verdicts et manquants réels.	<i>(cmp_df, real_missing_df, stats dict)</i>	validation Quiver (House)
<code>reconcile</code>	Dédoublonne Quiver, apparie par clé bioguide×ticker×date×sens, mesure accords sens/date/montant et deltas par membre.	<i>dict de DataFrames (reco, accords, deltas)</i>	validation Quiver (Sénat)

crosscheck.py — *Triangulation multi-sources et statut de validation par déposant ; matérialise que le papier OCR est source unique.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_norm_name</code>	Normalise un nom : minuscules, ne garde que lettres et espaces, compresse les espaces multiples.	<i>str : nom normalisé pour appariement</i>	utilitaire (normalisation nom)
<code>per_filer_status</code>	Agrège par bioguide nos comptes total/OCR/digital, joint Quiver, puis attribue un statut (quiver_validable, ocr_unique, digital).	<i>DataFrame : statut trié par volume</i>	qualité (statut déposant)
<code>load_kadoa_house</code>	Lit le JSON Kadoa, garde les entrées chambre house, mappe nom normalisé vers trade_count.	<i>dict : nom normalisé -> nombre de trades</i>	triangulation (Kadoa)
<code>load_hsw_counts</code>	Lit le JSON House Stock Watcher et compte les transactions par représentant (nom normalisé).	<i>dict : nom normalisé -> nombre de transactions</i>	triangulation (HSW)
<code>add_external_counts</code>	Joint par nom normalisé les comptes Kadoa et HSW au tableau de statut comme colonnes informatives.	<i>DataFrame : statut enrichi des comptes externes</i>	triangulation (fusion externe)
<code>summary</code>	Regroupe le tableau de statut par statut et somme déposants, transactions et part OCR.	<i>DataFrame : récapitulatif par statut</i>	qualité (récapitulatif)

sector_enrich.py — *Enrichissement secteur : ticker → GICS → ETF SPDR, hybride yfinance + repli/audit LLM + overrides manuels.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_cache</code>	Lit le JSON de cache du chemin ; renvoie les mappings seulement si pipeline_sha et model concordent, sinon dict vide.	<i>dict des mappings ticker→infos (vide si invalide)</i>	ticker/cache
<code>save_cache</code>	Écrit le cache JSON (model, pipeline_sha, mappings) au chemin donné, indentation 1, sans échappement Unicode.	— (effet de bord : écrit le fichier cache)	ticker/cache
<code>_yf_sector_one</code>	Importe yfinance, lit le champ sector d'un ticker, le traduit en GICS via la table ; None si erreur ou absent.	<i>str secteur GICS ou None</i>	secteur (yfinance)
<code>resolve_yfinance</code>	Interroge yfinance ticker par ticker sur plusieurs passes, ré-essaye les None avec pauses croissantes.	<i>dict ticker→secteur GICS (ou None)</i>	secteur (yfinance)

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_map_sectors_batch</code>	Appelle l'API Claude avec outil forcé <code>map_sectors</code> , ré-essaie avec back-off sur erreurs réseau ; extrait les mappings.	<i>liste de mappings (dicts ticker/secteur)</i>	secteur (LLM)
<code>resolve_llm</code>	Sans clé API renvoie vide ; sinon batche les tickers vers le LLM, ne garde le secteur que si coté et valide.	<i>dict ticker→secteur GICS validé ou None</i>	secteur (LLM)
<code>_norm_ticker</code>	Convertit en chaîne, retire les espaces et met en majuscules.	<i>str ticker normalisé</i>	utilitaire
<code>resolve_sectors</code>	Filtre/normalise les tickers valides, calcule les manquants, croise yfinance puis LLM, choisit le final, sauvegarde le cache.	<i>dict cache complet ticker→infos secteur</i>	secteur (orchestration)
<code>enrich_sectors</code>	Copie le df, résout les secteurs, ajoute colonnes <code>sector_gics/etf_proxy/sector_source</code> , applique les overrides manuels par ticker.	<i>DataFrame enrichi (3 colonnes ajoutées)</i>	secteur (enrichissement)
<code>build_audit</code>	Pour chaque ticker du df présent au cache, construit une ligne d'audit yfinance vs LLM avec drapeau d'accord.	<i>DataFrame d'audit secteur croisé</i>	secteur (audit)

reporting.py — Sorties partagées : convention de nommage des tables, drapeaux QA, dashboard idempotent, export Excel multi-onglets.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>qa_flags</code>	Parcourt chaque ligne, repère les champs obligatoires vides ou « nan », accumule une ligne d'anomalie listant ces champs.	<i>DataFrame des anomalies (champs manquants)</i>	qualité
<code>upsert_status</code>	Construit un DataFrame, relit le CSV existant, retire les lignes de même clé puis concatène et réécrit (idempotent).	<i>DataFrame du dashboard mis à jour</i>	dashboard
<code>write_excel</code>	Ouvre un classeur openpyxl, écrit un onglet LISEZMOI optionnel puis un onglet par DataFrame (placeholder si vide).	— (effet de bord : écrit le fichier Excel)	sorties (Excel)

paths.py — Bootstrap des chemins et secrets ; DataRoot route les I/O sous `data/{chambre}/` par chambre.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_env</code>	Charge le <code>.env</code> du <code>base_dir</code> ; si <code>ANTHROPIC_API_KEY</code> manque encore, charge un <code>.env</code> trouvé ailleurs dans l'arbre.	— (effet de bord : variables d'environnement chargées)	utilitaire (secrets)
<code>get_secret</code>	Renvoie la variable d'environnement ; sinon parse ligne à ligne le <code>.env</code> local pour la clé demandée.	<i>str valeur du secret ou None</i>	utilitaire (secrets)
<code>DataRoot.__init__</code>	Stocke le nom de chambre et le <code>data_dir</code> converti en Path.	— (effet de bord : initialise l'instance)	utilitaire (chemins)
<code>DataRoot.for_chamber</code>	Fabrique de classe : construit un DataRoot pointant sur <code>repo_root/data/{chambre}</code> .	<i>instance DataRoot</i>	utilitaire (chemins)
<code>DataRoot.tables</code>	Propriété renvoyant le sous-dossier tables de la chambre.	<i>Path du dossier tables</i>	utilitaire (chemins)
<code>DataRoot.tables_dir</code>	Renvoie <code>tables/{année}</code> si une année est fournie, sinon le dossier tables racine.	<i>Path du dossier tables (par année)</i>	utilitaire (chemins)
<code>DataRoot.pdfs</code>	Propriété renvoyant le sous-dossier pdfs de la chambre.	<i>Path du dossier pdfs</i>	utilitaire (chemins)
<code>DataRoot.index</code>	Propriété renvoyant le sous-dossier index de la chambre.	<i>Path du dossier index</i>	utilitaire (chemins)
<code>DataRoot.reference</code>	Propriété renvoyant le sous-dossier reference de la chambre.	<i>Path du dossier reference</i>	utilitaire (chemins)

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
DataRoot.ocr_cache	Propriété renvoyant le sous-dossier ocr_cache de la chambre.	Path du dossier ocr_cache	utilitaire (chemins)
DataRoot.reports	Propriété renvoyant le sous-dossier reports de la chambre.	Path du dossier reports	utilitaire (chemins)
DataRoot.media	Propriété renvoyant le sous-dossier media sous reports.	Path du dossier media	utilitaire (chemins)
DataRoot.quiver_cache_path	Propriété construisant le chemin du CSV de cache Quiver propre à la chambre sous tables.	Path du fichier cache Quiver	utilitaire (chemins)

pipeline.py — Pipeline end-to-end unifié : orchestre par sous-processus les étapes digital/OCR/fusion des deux chambres.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
parse_years	Parse la spec d'années : intervalle « a-b » en range, sinon liste « x,y » ou année unique en entiers.	list[int] des années	orchestration (CLI)
build_steps	Assemble la séquence ordonnée (label, args-module) selon skip_ocr/force/no_quiver/mode, ajoute l'enrichissement ancienneté final.	list de tuples (label, args-module)	orchestration
main	Parse les arguments CLI, construit les étapes, les imprime puis lance chaque module en sous-processus, arrêt au premier échec.	— (effet de bord : exécute le pipeline, sys.exit si échec)	orchestration

quality.py — Rapport qualité Ramify : charge les FINAL, calcule cinq contrôles, génère figures et RAP-PORT_QUALITE.md.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
_final_path	Construit le chemin de la table FINAL selon la chambre (House sous tables/{année}, Sénat sous {année}).	Path de la table FINAL	chargement (chemins)
op_class	Met en minuscules l'operation_type et classe en buy/sell/exchange/other selon les mots-clés présents.	str classe canonique d'opération	qualité (normalisation)
load_final	Concatène tous les FINAL des deux chambres, ajoute file_year/txn_year, parse les dates, calcule lag_days, midpoint et op.	DataFrame consolidé enrichi	chargement/fusion
date_coherence	Par chambre : compte dates parseables, part cohérente (lag≥0), incohérentes, années de transaction implausibles, dates manquantes.	DataFrame résumé de cohérence des dates	qualité (a) dates
delay_buckets	Par chambre : ventile le délai de divulgation en tranches ≤45j/45-75j/>75j/négatif et la médiane positive.	DataFrame ventilation des délais	qualité (b) délai
delay_outliers	Filtre les délais au-dessus du seuil, trie décroissant, garde le top et un sous-ensemble de colonnes.	DataFrame des divulgations les plus tardives	qualité (b) délai
amount_distribution	Calcule stats descriptives globales et par chambre du midpoint, plus le top 15 déposants par volume estimé.	dict {overall, by_chamber, top_volume}	qualité (c) montants
coverage_per_member	Appelle crosscheck.per_filer_status par chambre, fusionne nombre d'années actives et première/dernière année de transaction.	DataFrame de couverture par membre	qualité (d) couverture
eligible_members	Filtre les membres avec au moins min_trades transactions ; compte éligibles, total et éligibles actifs ≥3 ans.	dict de comptes d'éligibilité	qualité (d) couverture
unmatched_purchase_rate	Apparie chaque achat (bioguide+ticker) à une vente ultérieure ; calcule par chambre la part d'achats sans sortie déclarée.	DataFrame du taux d'achats non appariés	qualité (e) sorties

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_figures</code>	Avec matplotlib Agg, génère quatre PNG (délai, montants log, transactions par an, top déposants) dans le dossier.	<i>list des chemins de figures écrites</i>	qualité (figures)
<code>_md_table</code>	Formate un DataFrame en table Markdown manuelle, échappant les barres et vidant les valeurs nulles.	<i>str table Markdown</i>	rapport (Markdown)
<code>build_report</code>	Charge les données, calcule les cinq contrôles et figures, assemble le Markdown puis écrit RAPPORT_QUALITE.md.	<i>Path du rapport Markdown écrit</i>	rapport (Markdown)
<code>main</code>	Ancre le repo_root, appelle build_report et imprime les chemins du rapport et des figures.	— (<i>effet de bord : génère le rapport</i>)	orchestration

enrich_tenure.py — *Passe post-FINAL ajoutant years_in_office aux deux chambres, offline et byte-à-byte préservante.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>final_files</code>	Génère (chambre, année, chemin) pour chaque table FINAL existante, House sous tables/ puis Sénat sous {année}.	<i>itérateur de tuples (chambre, année, Path)</i>	ancienneté (découverte)
<code>_fmt</code>	Renvoie chaîne vide si None ou NaN, sinon l'entier converti en chaîne.	<i>str valeur formatée</i>	utilitaire
<code>enrich_file</code>	Lit le brut, saute si colonne déjà présente, vérifie la cohérence lignes/enregistrements, append years_in_office en préservant les octets.	<i>bool True si enrichi, False si déjà fait</i>	ancienneté (enrichissement)
<code>main</code>	Pour chaque FINAL, charge la référence de chambre (mise en cache), enrichit le fichier et imprime un récapitulatif des comptes.	— (<i>effet de bord : enrichit les fichiers FINAL</i>)	orchestration

Orchestrateur Chambre — house/ (3 modules, 48 fonctions)

digital.py — *Pipeline piste digitale House multi-années : parse PTR lisibles, identité, dédup, cross-check baseline et validation Quiver.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_dotenv</code>	Repli défini seulement si python-dotenv absent : ne fait rien et renvoie False.	<i>False (stub si dotenv manquant)</i>	utilitaire
<code>build_reference</code>	Charge le référentiel via congress_core.identity et peuple les globals identité (univers, index noms, comités, matcher).	— (<i>effet de bord : peuple globals, imprime</i>)	identité
<code>_find_continuation</code>	Cherche un code actif [XX] suivi d'un montant dans une ligne ; reformate un préfixe tiret en parenthèses.	<i>tuple (code_actif, second_montant, préfixe)</i>	parsing
<code>_join_split_lines</code>	Recolle les transactions éclatées sur 2 ou 3 lignes en réinsérant code actif et second montant au bon endroit.	<i>liste de lignes recollées</i>	parsing
<code>_is_txn_start</code>	Vérifie qu'une ligne commence une transaction : motif TXN au début ou précédé d'un code actif.	<i>booléen (début de transaction)</i>	parsing
<code>parse_ptr</code>	Parse le texte PTR : recolle lignes, détecte transactions, agrège blocs descriptifs, extrait ticker, type actif, propriétaire, montant.	<i>liste de dicts transactions</i>	parsing
<code>match_bioguide</code>	Délègue au matcher du cœur peuplé par build_reference ; renvoie None si matcher absent.	<i>identifiant bioguide ou None</i>	identité
<code>load_ptr_index</code>	Lit l'index XML de l'année, construit un DataFrame, filtre FilingType=P dans la fenêtre de divulgation, ajoute l'URL PDF.	<i>(DataFrame PTR filtré, distribution filing_type)</i>	parsing

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>resolve_pdf_path</code>	Construit le chemin du PDF par année et <code>doc_id</code> ; renvoie le Path s'il existe, sinon <code>None</code> .	<i>Path du PDF ou None</i>	utilitaire
<code>extract_text</code>	Ouvre le PDF avec <code>pdfplumber</code> et concatène le texte de toutes les pages ; renvoie vide en cas d'erreur.	<i>texte du PDF (str)</i>	parsing
<code>n_pages</code>	Ouvre le PDF avec <code>pdfplumber</code> et renvoie le nombre de pages ; <code>None</code> en cas d'erreur.	<i>nombre de pages ou None</i>	utilitaire
<code>build_manifest</code>	Lit chaque PDF en place, extrait le texte, route en lisible/non_lisible/absent, écrit le manifeste CSV.	<i>(textes par doc, DataFrame manifeste)</i>	OCR routage
<code>parse_docs</code>	Parse chaque texte lisible, accumule lignes et échecs, écrit les échecs CSV, calcule le rendement.	<i>(lignes, échecs, nb lisibles, taux rendement)</i>	parsing
<code>join_identity</code>	Fusionne lignes parsées avec l'index, résout bioguide, mappe parti, comités, drapeau commission clé, chambre.	<i>DataFrame enrichi identité</i>	identité
<code>_legacy_key</code>	Délègue au cœur partagé pour calculer le hash de clé naturelle House d'une ligne.	<i>hash de clé naturelle (str)</i>	dédup/schéma
<code>finalize</code>	Normalise champs, calcule clé naturelle et indice d'occurrence, déduplique en gardant la divulgation la plus récente.	<i>(table canonique, table complète, stats dédup)</i>	dédup/schéma
<code>_sem1_year</code>	Charge en cache la baseline historique et filtre les lignes dont l'année de divulgation égale l'année demandée.	<i>DataFrame baseline filtré par année</i>	validation baseline
<code>crosscheck_semaine1</code>	Compare comptes par doc entre nous et la baseline, détecte dérives (baseline>0 et nous=0), écrit le CSV.	<i>dict métriques de cross-check</i>	validation baseline
<code>fetch_quiver</code>	Charge le cache Quiver House si présent, sinon appelle l'API live avec token, filtre la chambre, met en cache.	<i>DataFrame Quiver ou None</i>	validation Quiver
<code>validate_quiver</code>	Compare comptes par déclarant per-lot et recouvrement au niveau transaction vs Quiver, distingue manques papier, écrit CSV.	<i>dict métriques de validation ou None</i>	validation Quiver
<code>run_year</code>	Orchestre une année : index, manifeste, parsing, identité, finalisation, cross-check, Quiver, backlog OCR ; écrit fichiers.	<i>(résumé, table finale, backlog)</i>	orchestration
<code>main</code>	Analyse les arguments, construit le référentiel, lance <code>run_year</code> par année, fusionne et écrit statut et backlog cumulés.	<i>— (effet de bord : écrit CSV, imprime)</i>	orchestration

ocr.py — OCR multi-années des PTR scannés via Claude Vision : deskew, extraction outillée, cache, ticker LLM, fusion finale et validation Quiver.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_load_ticker_cache</code>	Lit le cache JSON ticker LLM ; ne renvoie les mappings que si <code>prompt_sha</code> et modèle concordent.	<i>dict des mappings nom→ticker</i>	ticker
<code>_save_ticker_cache</code>	Écrit le cache ticker LLM en JSON avec modèle, <code>prompt_sha</code> et mappings.	<i>— (effet de bord : écrit le cache JSON)</i>	ticker
<code>_map_tickers_batch</code>	Appelle l'API outil <code>map_tickers</code> sur un lot de noms avec retries à backoff ; extrait les mappings du bloc <code>tool_use</code> .	<i>liste de mappings ou exception</i>	ticker
<code>llm_resolve_tickers</code>	Pour les lignes sans ticker, interroge le LLM par lots de 60, met en cache, remplit ticker et marque la source llm.	<i>DataFrame avec tickers complétés</i>	ticker
<code>detect_rotation</code>	Montre la page aux 4 rotations au modèle, lui fait choisir celle droite, renvoie l'angle horaire ; 0 si échec.	<i>angle horaire à appliquer (int)</i>	OCR Vision
<code>rotate_b64_png</code>	Décode le PNG base64 et le tourne de l'angle horaire (PIL anti-horaire donc négatif) ; ré-encode en base64.	<i>PNG base64 tourné (str)</i>	OCR Vision

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
pdf_to_b64_images	Rend chaque page en PNG base64 ; si skew détecte et redresse page par page, renvoie images et angles.	<i>images ou (images, angles)</i>	OCR Vision
OcrError	Classe d'exception dédiée aux erreurs d'extraction OCR ; corps vide.	— (<i>type d'exception</i>)	utilitaire
_call_vision_tool	Envoie les images et le prompt à Claude avec l'outil record_transactions, retries selon code HTTP ; extrait les transactions.	<i>liste de transactions ou OcrError</i>	OCR Vision
extract_from_pdf	Extraction avec cache versionné et reprise par lot : ne refait que les lots en erreur, écrit le cache JSON.	<i>(transactions, objet cache)</i>	OCR Vision
natural_key_hash	Délègue au cœur partagé pour calculer le hash de clé naturelle House d'une ligne.	<i>hash de clé naturelle (str)</i>	dédup/schéma
date_confidence	Calcule le délai dépôt-transaction ; renvoie plausible si entre 0 et 75 jours, sinon implausible.	<i>'plausible' ou 'implausible' (str)</i>	qualité
normalize	Mappe code montant, propriétaire, dates ; construit la ligne au schéma, calcule confiance de date et clé naturelle.	<i>dict ligne normalisée</i>	dédup/schéma
_infer_asset_type	Délègue au cœur l'inférence du type d'actif depuis la description, contexte House.	<i>type d'actif (str ou None)</i>	secteur
run_ocr_year	Orchestre l'OCR d'une année : sélectionne scannés (exclut cluster C sauf filers prioritaires), extrait en parallèle, enrichit, fusionne finale.	<i>dict résumé ou None</i>	orchestration
validate_ocr_quiver	Compare comptes OCR vs Quiver par déclarant fenêtré par année, attribue un verdict, écrit le CSV.	<i>dict décompte des verdicts ou None</i>	validation Quiver
main	Analyse arguments, vérifie la clé API, construit le référentiel, lance OCR par année puis validation Quiver.	— (<i>effet de bord : imprime, écrit fichiers</i>)	orchestration

echantillon.py — OCR sur échantillon représentatif par année (~70 docs) : prompt durci, skew, enrichissement ticker et comparaison Quiver par cluster et année.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
_route	Choisit DPI et pages-par-appel selon la densité : doc dense renvoie DPI haut et 1 page, sinon DPI moyen et 3.	<i>tuple (dpi, pages_par_appel)</i>	OCR Vision
_render	Rend les pages (plafonnées au cap) en PNG base64 ; avec client détecte et redresse l'orientation page par page.	<i>(images, angles appliqués)</i>	OCR Vision
_call	Envoie les images et le prompt durci à Claude avec l'outil record_transactions, retries à backoff ; extrait les transactions.	<i>liste de transactions</i>	OCR Vision
_ocr_doc	OCR d'un doc échantillon : sert le cache durci versionné sinon ré-OCR redressé ; ne lève jamais, renvoie liste vide si échec.	<i>(doc_id, transactions)</i>	OCR Vision
_global_ticker_dict	Agrège toutes les tables digitales 2020-2026 en dictionnaire nom normalisé→ticker (première valeur retenue).	<i>dict nom normalisé→ticker</i>	ticker
run_echantillon	Orchestre l'OCR de l'échantillon (exclut cluster C), normalise, enrichit ticker et bioguide, écrit les résultats CSV.	<i>DataFrame des transactions OCR</i>	orchestration
_nearest_signed	Trouve la date Quiver la plus proche d'une date donnée et calcule l'écart signé en jours.	<i>(date proche, écart signé) ou (None, None)</i>	validation Quiver
compare_quiver	Calcule par doc la précision ticker et ticker+date (tolérance date) contre Quiver fenêtré ; écrit le CSV par doc.	<i>DataFrame précision par doc</i>	validation Quiver

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>detailed_quiver_diff</code>	Diff transaction par transaction vs Quiver : statut match, date fausse, ticker faux, sans données ou sans ticker ; écrit CSV.	<i>DataFrame diff par transaction</i>	validation Quiver

Orchestrateur Sénat — `senate/` (10 modules, 79 fonctions)

digital.py — Pipeline Sénat digital multi-années : scraping eFD des PTR électroniques, parsing, enrichissement, validation Quiver par an.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>accept_agreement</code>	GET la page d'accueil eFD, lit le csrftoken, POST l'agrément de prohibition avec le token.	<i>bool : sessionid ou csrftoken présent dans les cookies</i>	acquisition
<code>fetch_ptr_list</code>	POST paginé l'endpoint data eFD (<code>report_types[11]</code>), extrait nom/kind/uuid/date par regex, filtre sur la fenêtre.	<i>DataFrame des dépôts PTR de la fenêtre</i>	acquisition
<code>fetch_report</code>	Renvoie le HTML du PTR depuis le cache disque s'il existe, sinon GET, écrit le cache, pause.	<i>(url, texte HTML) ou (url, chaîne vide)</i>	acquisition
<code>amount_midpoint</code>	Extrait les nombres dollar par regex ; moyenne des deux premiers, sinon valeur unique, sinon None.	<i>float milieu de fourchette, ou None</i>	parsing
<code>norm_op</code>	Normalise le type d'opération par préfixe/mot-clé : Purchase, Exchange, Sale (Partial/Full), Sale.	<i>str type d'opération normalisé</i>	parsing
<code>_find_col</code>	Parcourt les colonnes, renvoie la première dont le nom minuscule contient tous les mots-clés.	<i>nom de colonne trouvé, ou None</i>	utilitaire
<code>parse_electronic</code>	Lit les tables HTML, repère celle des transactions, localise chaque colonne, construit les lignes de transactions.	<i>liste de dicts transactions</i>	parsing
<code>run_year</code>	Récupère dépôts d'une année, parse les PTR électroniques, enrichit, écrit CSV, réconcilie Quiver, calcule verdicts et statut.	<i>dict statut de l'année, ou None</i>	orchestration annuelle
<code>run_year.verdict</code>	Classe une ligne de delta : quiver_seul, concordant, quiver_sans_donnee, nous_plus ou quiver_plus_a_verifier.	<i>str verdict de comparaison</i>	validation Quiver
<code>main</code>	Parse les années, charge référentiel et cache Quiver, agréé la session, boucle <code>run_year</code> , agrège et écrit le dashboard.	<i>— (effet de bord : écrit 00_year_status.csv)</i>	orchestration

ocr.py — Orchestrateur OCR papier multi-années : découvre les PTR papier, les OCR-ise via le moteur figé, enrichit et écrit par an.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_image_b64_safe</code>	Cache l'image par hash SHA1 de l'URL complète (anti-collision), retries, redimensionne $\leq$ LONG_EDGE, renvoie PNG base64.	<i>str image PNG encodée en base64</i>	OCR Vision
<code>accept_agreement</code>	GET l'accueil eFD, lit le csrftoken, POST l'agrément de prohibition avec le token.	<i>bool : sessionid ou csrftoken présent dans les cookies</i>	acquisition
<code>fetch_ptr_list</code>	POST paginé l'endpoint data eFD, extrait nom/kind/uuid/date par regex, filtre sur la fenêtre de dates.	<i>DataFrame des dépôts PTR de la fenêtre</i>	acquisition
<code>fetch_paper_html</code>	Renvoie le cache HTML s'il contient les images, sinon GET avec retries, ré-agrège la session si expirée.	<i>bool : HTML papier disponible avec images</i>	acquisition
<code>discover_index</code>	Agréé, parcourt les années, filtre les PTR papier, collecte uuid/membre/date, écrit l'index CSV.	<i>DataFrame index des rapports papier</i>	acquisition

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>ocr_one</code>	Fetch le HTML papier puis délègue l'extraction Vision au moteur figé <code>so.extract_report</code> .	<i>(liste transactions, dict cache) du rapport</i>	OCR Vision
<code>build_table</code>	OCR-ise chaque rapport, écarte les lignes-exemple, normalise, résout tickers et identité, applique le schéma.	<i>(df, per_doc, failures, n_example)</i>	OCR Vision
<code>build_table._resolve</code>	Garde le ticker explicite ; sinon tente <code>recover_ticker</code> sur la description ; sinon <code>None</code> .	<i>(ticker, source) tuple</i>	ticker
<code>qa</code>	Affiche un contrôle qualité : compte identité/ticker, dates dans la fenêtre déclarée, owner, montants, <code>asset_type</code> .	— <i>(effet de bord : impression console)</i>	qualité
<code>main</code>	Parse les arguments, vérifie la clé API, construit l'index, exécute le mode <code>index/pilote/full</code> , écrit les CSV.	— <i>(effet de bord : écrit CSV OCR par an)</i>	orchestration

ocr_engine.py — Moteur OCR figé des PTR Sénat scannés : récupère les images, appelle Claude Vision avec `cache versionné`, `normalise`.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_api_key</code>	Lit <code>ANTHROPIC_API_KEY</code> dans l'environnement, sinon parse le fichier <code>.env</code> local à la recherche de la clé.	<i>str clé API, ou None</i>	utilitaire
<code>report_gif_urls</code>	Lit le HTML caché du rapport et renvoie toutes les URLs d'images <code>.gif</code> via <code>regex</code> .	<i>liste d'URLs .gif</i>	OCR Vision
<code>_image_b64</code>	Récupère le <code>.gif</code> (cache disque d'abord, sinon <code>GET</code>), redimensionne $\leq$ <code>LONG_EDGE</code> , renvoie PNG <code>base64</code> .	<i>str image PNG encodée en base64</i>	OCR Vision
<code>OcrError</code>	Classe d'exception dédiée signalant un échec d'extraction Vision (corps vide, hérite d' <code>Exception</code>).	— <i>(classe d'exception)</i>	utilitaire
<code>_call_vision</code>	Construit le contenu <code>image+prompt</code> , appelle l'API avec <code>tool</code> forcé, retries sur erreurs, renvoie les transactions du <code>tool_use</code> .	<i>liste de transactions extraites</i>	OCR Vision
<code>extract_report</code>	Sert le cache versionné si valide, sinon calcule la fenêtre, OCR-ise les images par lots, écrit le cache JSON.	<i>(liste transactions, dict cache)</i>	OCR Vision
<code>_infer_asset_type</code>	Déduit l' <code>asset_type</code> par motifs <code>regex</code> : <code>Municipal Security</code> , <code>Corporate Bond</code> , <code>Other</code> , <code>Stock</code> , sinon <code>None</code> .	<i>str type d'actif déduit, ou None</i>	secteur
<code>_fix_year</code>	Si la date lue tombe hors fenêtre PTR, tente de corriger l'année (année de dépôt ou précédente).	<i>str date ISO corrigée, ou entrée brute</i>	qualité
<code>normalize</code>	Mappe le code montant en fourchette, nettoie un préfixe propriétaire résiduel, corrige l'année, renvoie la ligne au schéma.	<i>dict transaction normalisée</i>	dédup/schéma
<code>_natural_key</code>	Concatène sept champs métier de la ligne et renvoie leur hash <code>SHA256</code> .	<i>str hash naturel SHA256</i>	dédup/schéma
<code>main</code>	OCR-ise les 4 PTR <code>Blumenthal Q1</code> , écarte exemples, normalise, résout tickers et identité, écrit le CSV <code>golden</code> .	<i>DataFrame OCR (effet de bord : écrit CSV)</i>	orchestration
<code>main._resolve</code>	Garde le ticker explicite ; sinon tente <code>recover_ticker</code> ; sinon <code>None</code> avec source <code>'none'</code> .	<i>(ticker, source) tuple</i>	ticker

fusion.py — Fusion non destructrice `digital+OCR` par `an` puis enrichissement global `ticker/secteur/date`, écrit les tables `FINALE`.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>date_confidence</code>	Calcule le délai dépôt–transaction ; renvoie plausible (0–75 j), implausible, ou unknown si dates manquantes.	<i>str niveau de confiance de la date</i>	qualité
<code>main</code>	Détecte les années, fusionne digital+OCR sans destruction, enrichit le corpus (ticker/secteur/date), écrit FINAL et dashboard.	— (effet de bord : écrit <i>FINAL.csv</i> + <i>00_final-status.csv</i>)	fusion

identity.py — *Référentiel bioguide, matcher d'identité par chambre, et enrichissement déterministe (ticker, options, dédup) au schéma 23 colonnes du Sénat.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>strip_accents</code>	Décompose en NFKD et retire les caractères combinants pour ôter les accents.	<i>str sans accents</i>	utilitaire normalisation
<code>norm</code>	Ôte accents, passe en minuscules, remplace tout non-lettre par espace, compacte les espaces.	<i>str normalisé en minuscules</i>	utilitaire normalisation
<code>split_name</code>	Prend la partie avant la virgule, écarte les suffixes, renvoie dernier, premier, tokens du milieu.	<i>tuple (last, first, [middles])</i>	identité
<code>load_reference</code>	Charge les YAML législateurs courants/historiques, construit index nom→bioguide et commissions, calcule le flag clé.	<i>tuple de 6 dictionnaires/index</i>	identité référentiel
<code>make_matcher</code>	Fabrique une closure match : override manuel, exact, puis chambre Sénat, titulaire, désambiguïsation prénom.	<i>fonction match(declarant)→ bioguide ou None</i>	identité
<code>recover_ticker</code>	Teste si la description correspond à un motif ticker via regex et renvoie le symbole capturé.	<i>str ticker ou None</i>	ticker
<code>natural_key</code>	Concatène sept champs métier et renvoie leur SHA-256 stable, sans inclure le ticker.	<i>str hash hexadécimal SHA-256</i>	dédup/schéma
<code>enrich</code>	Mappe identité, enrichit ticker, reclasse options, normalise dates, calcule <code>natural_key</code> , <code>occurrence_index</code> , déduplique cross-rapport, réindexe au schéma.	<i>DataFrame au schéma 23 colonnes</i>	identité enrichissement

ticker_resolve.py — *Résolution du ticker nom→symbole pour le Sénat en trois étapes : explicite, dictionnaire électronique, puis passe LLM Claude.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_clean_name</code>	Coupe la description au premier double-espace suivi d'un mot-clé méta via regex, puis trim.	<i>str nom nettoyé</i>	ticker normalisation
<code>_norm_asset</code>	Nettoie, majuscule, retire ticker explicite, ponctuation et suffixes corporatifs, compacte, applique corrections OCR.	<i>str clé d'actif normalisée</i>	ticker normalisation
<code>_explicit_ticker</code>	Cherche un ticker entre parenthèses/tiret en fin de description, vérifie longueur 1 à 5.	<i>str ticker ou None</i>	ticker
<code>_api_key</code>	Charge le .env du dépôt puis le .env courant, renvoie la variable ANTHROPIC_API_KEY.	<i>str clé API (ou vide)</i>	utilitaire LLM
<code>_load_cache</code>	Lit le cache JSON et renvoie ses mappings seulement si <code>prompt_sha</code> et <code>model</code> concordent.	<i>dict mappings ou dictionnaire vide</i>	ticker cache
<code>_save_cache</code>	Crée le dossier parent et écrit <code>model</code> , <code>prompt_sha</code> et mappings en JSON.	— (effet de bord : écrit le cache JSON)	ticker cache
<code>_map_tickers_batch</code>	Appelle l'API Claude avec <code>tool_use</code> forcé, réessaie avec backoff sur erreurs réseau, extrait mappings.	<i>list de mappings nom→ticker</i>	ticker LLM
<code>llm_resolve_tickers</code>	Cible lignes sans ticker, interroge le LLM par lots de 60 (cache versionné), remplit <code>ticker_source=llm</code> .	<i>DataFrame avec tickers LLM remplis</i>	ticker LLM

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>resolve_tickers</code>	Normalise vacuité, corrige label OCR, applique dictionnaire électronique puis passe LLM, journalise les gains.	<i>DataFrame avec ticker/ticker_source résolus</i>	ticker orchestration

sector_enrich.py — *Enrichissement secteur GICS et ETF SPDR par jointure sur le ticker : yfinance primaire, repli LLM, overrides manuels, audit croisé.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_cache</code>	Lit le cache secteur JSON et renvoie ses mappings seulement si pipeline_sha et model concordent.	<i>dict mappings ticker→secteur</i>	secteur cache
<code>save_cache</code>	Écrit model, pipeline_sha et mappings du secteur en JSON sur disque.	— (effet de bord : écrit <i>sector_cache.json</i>)	secteur cache
<code>_yf_sector_one</code>	Interroge yfinance pour un ticker et mappe le secteur Yahoo vers le GICS canonique.	<i>str secteur GICS ou None</i>	secteur yfinance
<code>resolve_yfinance</code>	Résout les tickers via yfinance en plusieurs passes, réessayant les None pour absorber le throttle Yahoo.	<i>dict ticker→secteur GICS</i>	secteur yfinance
<code>_map_sectors_batch</code>	Appelle l'API Claude avec tool_use forcé pour les secteurs, réessaie avec backoff, extrait mappings.	<i>list de mappings ticker→secteur</i>	secteur LLM
<code>resolve_llm</code>	Charge la clé API, interroge Claude par lots de 40, filtre is_listed et secteur valide.	<i>dict ticker→secteur GICS ou vide</i>	secteur LLM
<code>_norm_ticker</code>	Met le ticker en majuscules et supprime les espaces de bordure.	<i>str ticker normalisé</i>	utilitaire normalisation
<code>resolve_sectors</code>	Construit le référentiel : yfinance sur le manquant, LLM (audit complet ou repli), choisit final, sauve le cache.	<i>dict cache complet ticker→secteur</i>	secteur orchestration
<code>enrich_sectors</code>	Résout les tickers du DataFrame, ajoute sector_gics, etf_proxy, sector_source, applique les overrides manuels.	<i>DataFrame avec colonnes secteur ajoutées</i>	secteur application
<code>build_audit</code>	Pour chaque ticker distinct, construit une ligne d'audit avec source, yf, llm et accord croisé.	<i>DataFrame d'audit secteur par ticker</i>	secteur audit
<code>summary</code>	Imprime sources, taux d'accord croisé yfinance↔LLM, extrait de désaccords et distribution sectorielle.	— (effet de bord : <i>impressions console</i>)	secteur audit
<code>main</code>	Charge le CSV final, enrichit secteurs, le réécrit, construit l'audit 06f, imprime la couverture.	— (effet de bord : <i>réécrit CSV + audit</i>)	secteur orchestration CLI

quiver_audit.py — *Validation exhaustive Sénat ↔ Quiver en sept aspects (couverture, ticker, sens, dates, montant, deltas), en lecture seule.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>norm_ticker</code>	Majuscule, écarte valeurs vides/placeholders, retire suffixe option, remplace points et tirets par underscore.	<i>str clé instrument normalisée</i>	validation Quiver normalisation
<code>norm_sense</code>	Réduit l'opération à Purchase, Sale ou Exchange selon l'initiale en minuscules.	<i>str sens réduit</i>	validation Quiver normalisation
<code>_low_bound</code>	Extrait par regex la première borne dollar de la fourchette de montant.	<i>int borne basse ou None</i>	validation Quiver montant
<code>reconcile</code>	Déduplique amendements Quiver, restreint aux sénateurs communs tickérissables, calcule couverture, ticker, accords champ et deltas.	<i>dict de cinq DataFrames d'audit</i>	validation Quiver

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_load_inputs</code>	Charge FINAL et cache Quiver, filtre la fenêtre Q1, exclut les délégués hors-chambre.	<i>tuple (df, qwin)</i>	validation Quiver acquisition CLI
<code>main</code>	Charge les entrées, appelle reconcile, écrit les CSV d'audit, imprime couverture, accords et deltas.	— (<i>effet de bord : écrit tables audit + impressions</i>)	validation Quiver orchestration CLI

revalidate_quiver.py — *Re-validation Quiver hors-ligne par année : relit tables et cache, rejoue reconcile avec dédup amendements, met à jour le dashboard annuel.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>verdict</code>	Classe une ligne de delta en concordant, nous_plus, quiver_seul ou quiver_sans_donnee selon ses compteurs.	<i>str verdict de catégorisation</i>	validation Quiver classification
<code>main</code>	Pour chaque année, relit table et Quiver, rejoue reconcile, écrit 07c-f/07, agrège le dashboard de statut.	— (<i>effet de bord : réécrit artefacts + dashboard</i>)	validation Quiver orchestration

census_probe.py — *Phase 0 : recense les PTR Sénat 2020→2026 sur l'eFD, sonde le parser sur HTML ancien et compare l'ordre de grandeur à Quiver.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>accept_agreement</code>	GET l'accueil eFD, récupère le csrftoken, POST l'acceptation de l'accord, vérifie la présence des cookies.	<i>bool session ouverte</i>	acquisition eFD
<code>fetch_ptr_list</code>	Pagine la recherche eFD des PTR déposés dans la fenêtre, extrait nom, type, uuid, date, refiltre par date.	<i>DataFrame des PTR listés</i>	acquisition recensement
<code>fetch_report</code>	Renvoie le HTML du rapport depuis le cache disque, sinon le télécharge, le met en cache et le renvoie.	<i>tuple (url, html)</i>	acquisition eFD
<code>amount_midpoint</code>	Extrait les montants dollar et renvoie la moyenne des deux bornes, ou la valeur unique.	<i>float point milieu ou None</i>	parsing montant
<code>norm_op</code>	Normalise le type d'opération en Purchase, Exchange, Sale partielle/complète ou Sale.	<i>str opération normalisée</i>	parsing
<code>_find_col</code>	Renvoie la première colonne dont le libellé minuscule contient tous les mots-clés fournis.	<i>nom de colonne ou None</i>	utilitaire parsing
<code>parse_electronic</code>	Lit les tables HTML, repère la table transactions, localise les colonnes, construit les lignes ou un statut d'échec.	<i>tuple (rows, columns, status)</i>	parsing HTML
<code>main</code>	Accepte l'accord, recense par an, sonde le parser sur années anciennes, compare à Quiver, écrit CSV et récap.	— (<i>effet de bord : écrit CSV census/probe + impressions</i>)	orchestration Phase 0

12.2 Annexe B — les 44 fichiers .py en un coup d'œil

Fichier	Rôle
<code>congress_core/</code> (15, dont <code>__init__</code>)	
<code>schema.py</code>	clé naturelle (7 champs) + dédup non destructrice
<code>identity.py</code>	nom → bioguide + parti/commissions/ancienneté
<code>amounts.py</code>	fourchette → milieu, code → opération

Fichier	Rôle
tickers.py	symbole boursier + type d'actif
vision_ocr.py	OCR Claude Vision + cache versionné + deskew
llm_resolve.py	cache LLM versionné (nom→ticker)
quiver.py	validation externe (Chambre brute / Sénat normalisée)
crosscheck.py	statut par déposant (OCR-unique vs Quiver)
sector_enrich.py	ticker → GICS → ETF (yfinance + LLM)
reporting.py	conventions de sortie, QA, Excel
paths.py	chemins (DataRoot) + secrets
pipeline.py	l'orchestrateur (6 étapes + qualité)
quality.py	rapport de qualité (5 contrôles + figures)
enrich_tenure.py	years_in_office (append octet-préservant)
house/ (4, dont __init__)	
digital.py	PTR électroniques → 06..._transactions
ocr.py	scans → Vision → fusion → 06..._FINAL
echantillon.py	mesure stratifiée OCR (pilote/audit)
senate/ (11, dont __init__)	
digital.py	eFD électronique → 06..._transactions
ocr.py	papier eFD → Vision → 06b
ocr_engine.py	moteur OCR figé (Q1) réutilisé
fusion.py	fusion digital+OCR + enrichissement corpus → FINAL
identity.py	référentiel + matcher Sénat
ticker_resolve.py	ticker en 3 étapes (explicit/dict/LLM)
sector_enrich.py	secteur GICS→ETF (Sénat)
quiver_audit.py	réconciliation Quiver (07c-07f)
revalidate_	re-validation Quiver offline
quiver.py	
census_probe.py	Phase 0 : census + sondage parser
tests/regression/ (14)	
golden (build_golden/check_golden + Sénat) · reproductions (test_schema, test_identity, test_	
amounts_tickers, test_senate_repro, test_vision_sha, test_incremental, test_tenure, test_	
crosscheck) · audit_metrics · _apply_house_sector	

12.3 Annexe C — le schéma FINAL (28 colonnes)

#	Colonne	Sens	Remplie par
1	bioguide_id	identifiant officiel du membre	identity
2	declarant_name	nom tel que déposé	parse / OCR
3	chamber	house / senate	enrich_identity
4	party	parti	Reference.party
5	state_district	état + district	03_ptr_index
6	committee_membership	commissions (liste)	Reference.committees
7	committees_key_flag	commission « clé » ?	is_key_committee
8	transaction_date	date de la transaction	parse / OCR
9	disclosure_date	date de divulgation (dépôt)	03_ptr_index
10	ticker	symbole boursier	tickers / llm_resolve
11	asset_description	libellé de l'actif	parse / OCR
12	asset_type	type d'actif	infer_asset_type
13	sector_gics	secteur GICS	sector_enrich
14	etf_proxy	ETF SPDR du secteur	GICS_TO ETF

#	Colonne	Sens	Remplie par
15	operation_type	Purchase / Sale / ...	operation_type_ from_code
16	amount_range	fourchette déclarée	parse / OCR
17	amount_midpoint	milieu de la fourchette	amount_midpoint
18	amount_split_flag	lot multi-actifs ?	finalize
19	owner	propriétaire (self/spouse...)	normalize
20	doc_id	identifiant du PTR	03_ptr_index
21	source_url	URL de la source	03_ptr_index
22	natural_key_hash	clé de dédup (7 champs)	schema
23	occurrence_index	rang du lot intra-dépôt	add_occurrence_index
24	provenance	*-electronic / *-ocr	pipeline
25	date_confidence	plausible / implausible (75 j)	date_confidence
26	ticker_source	explicit / elec_dict / llm / none	tickers / llm
27	sector_source	yfinance / llm / manual	sector_enrich
28	years_in_office	ancienneté à la date du trade	enrich_tenure

L'ordre exact diffère légèrement entre Chambre et Sénat (*date_confidence* plus tôt au Sénat); le contenu garanti — les 12 champs « métier » — est identique.

12.4 Annexe D — valeurs vérifiées (recalculées depuis les FINAL)

Indicateur	Chambre	Sénat
Transactions FINAL	81 646	8 841
dont digital / OCR	32 676 / 48 970	7 161 / 1 680
Identité (bioguide_id)	99,99 % (256/275)	100 % (64/67)
ticker	85,3 %	71,4 %
sector_gics / etf_proxy	83,2 %	62,1 %
asset_type	91,1 %	97,3 %
committee_membership	86,6 %	65,6 %
years_in_office	100 %	100 %
Quiver (transaction-niveau)	85,9 %/an (9 vrais-absents)	98–100 %/an (0 vrai raté)
Golden (zéro écart)	108 fichiers	69 fichiers
Qualité (depuis quality.py)		
Cohérence des dates	99,8 %	100,0 %
Dépôts $\leq$ 45 j (médiane 28 j)	86,9 %	84,9 %
Montants — médiane / p75	8 000 \$ / 32 500 \$	
Déposants ($\geq$ 10 trades / $\geq$ 3 ans)	320 (206 / 150)	
Achats sans sortie	22,5 %	39,6 %

Total : 90 487 transactions, 7 ans (2020–2026). Concentration : Rohit Khanna $\approx$ 30 862 transactions (top 3 Chambre $\approx$ 92,4 % de l'OCR).

12.5 Annexe E — reproduire / lancer

```
# Installation
python3.12 -m venv .venv && ./venv/bin/pip install -e .

# Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence
./venv/bin/python -m congress_core.pipeline --years 2020-2026
./venv/bin/python -m congress_core.pipeline --years 2024 --dry-run

# Rapport de qualite (offline, lecture seule des FINAL)
# -> docs/RAPPORT_QUALITE.md + docs/quality/*.png
./venv/bin/python -m congress_core.quality
```

```
# Filet "zero changement" (doit afficher ZERO ECART)
./venv/bin/python tests/regression/check_golden.py      # Chambre, 108
./venv/bin/python tests/regression/senate_check_golden.py # Senat, 69
./venv/bin/python tests/regression/test_senate_repro.py # reproductions
```

Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.